PAPIT: Distilling Mid-Layer LLM Attention into a Training-Free Image-Token Pruner for LLaVA-1.5

Abstract

*We present **PAPIT-Distill**, a* 1.3*M-parameter MLP that distils LLaVA-1.5-7B's mid-layer attention into a single-pass image-token pruner. On TextVQA at* k=25% *retention (a* 10× *attention-FLOP reduction) it gains* +12 *points over random pruning (*36.0 *vs.* 23.7*), essentially recovering the unpruned baseline (*36.7*). On GQA and VQA v2 at* k=50% *it closes the gap to unpruned to within a single point. Scoring a* 576*-patch image takes* 0.43 *ms — ∼*150× *faster than its CLIP-GradCAM analogue and ∼*500× *faster than the two-pass oracle it learns from. En route we report a quantitative* negative result *that motivates the method: cross-dataset Spearman* $\rho \approx 0$ *between CLIP-derived saliency and LLaVA's own LLM attention, ruling out training-free CLIP-aligned scorers as a class.*

1. Introduction

LLaVA-1.5 [1] composes a CLIP ViT-L/14 vision encoder [2, 3] with a 7B-parameter language model. Each of the $24 \times 24 = 576$ image tokens participates in every self-attention layer [4], making inference cost quadratic in the number of visual tokens. For a typical VQA query fewer than half of these tokens are relevant.

A common intuition is that the user's prompt should be enough to choose which tokens to keep, *without* any additional training. We make this claim concrete: *can a training-free saliency signal, read from LLaVA's own CLIP-initialised vision tower, predict which patches the LLM relies on, well enough to beat a query-agnostic random baseline?*

We answer this in three stages. First (§3) we instantiate the idea as **PAPIT**, a pipeline that inserts a GradCAM-based top-k between the ViT and the MLP projector. Second (§5.2) we show that PAPIT only *matches* random pruning, and that a direct alignment diagnostic explains why: the CLIP-aligned scoring signal has $\rho \approx 0$ with the LLM's own attention. Third (§5.4) we run a multi-layer LLM-attention oracle as a positive control — mid-layer attention *does* beat random — and distil that signal into a 1.3M-param MLP that runs in single-pass, training-free deployment.

Contributions. (i) A clean training-free token-pruning pipeline for LLaVA-1.5 with 41–90% attention-FLOP reduction at $k \in [25\%, 75\%]$. **(ii)** A cross-dataset *negative result*: CLIP-derived patch saliency is statistically uncorrelated with LLaVA's LLM attention, ruling out an entire family of training-free prompt-aware methods. **(iii) PAPIT-Distill**, a small MLP distilled from mid-layer LLM attention that turns the negative result into a deployable single-pass scorer. Code at `github.com/yarikama/PAPIT`.

2. Related Work

ViT-side token pruning for image classification is studied in DynamicViT [5] and AdaViT [6], which insert a learned scorer in front of each ViT block. Token merging (ToMe) [7] replaces scoring with similarity-aware merging. Both lines of work assume access to ground-truth supervision for training the scorer.

LLM-side pruning for VLMs. FastV [8] prunes visual tokens *inside* the LLM by reading layer-2 attention and dropping low-attention tokens. SparseVLM [9] extends this with hybrid relevance/diversity strategies. Both require a forward pass through the LLM before pruning.

Dense CLIP signals. MaskCLIP [10] shows that with a small architectural fix, patch-level CLIP features can be used for dense labels. Recent work [11] also documents that ViTs allocate register-like roles to a subset of patch tokens, which complicates naive top-k selection.

PAPIT differs from FastV in that pruning happens *before* the LLM sees the image, using a query-conditioned scoring step. The negative result we report (§5.2) shows why a naive instantiation of this idea fails; the distillation we propose (§5.4) is the smallest viable fix.

3. Method

3.1. PAPIT pipeline

Figure 1 shows the three-stage pipeline. The image is encoded by LLaVA's frozen vision tower (a CLIP ViT-L/14, no second encoder is loaded) producing 576 patch features $H \in \mathbb{R}^{576 \times 1024}$. The prompt is encoded by the matching CLIP text encoder into a single embedding $T \in \mathbb{R}^{768}$. A scoring step (§3.2 or §3.3) returns a vector $s \in \mathbb{R}^{576}$; the top-k indices select k patches that, together with one *anchor* token (the mean of all 576 patches, preserving global context), are passed through LLaVA's MLP projector and into the LLM. Position IDs are remapped to a dense $0, \ldots, k$ sequence to keep the LLM's positional encoding well-conditioned.

3.2. Scorer 1: GradCAM on LLaVA's vision tower

CLIP's contrastive objective only aligns the CLS token with text, so naive patch–text cosine similarity is unstable across retention ratios. We instead compute a Grad-

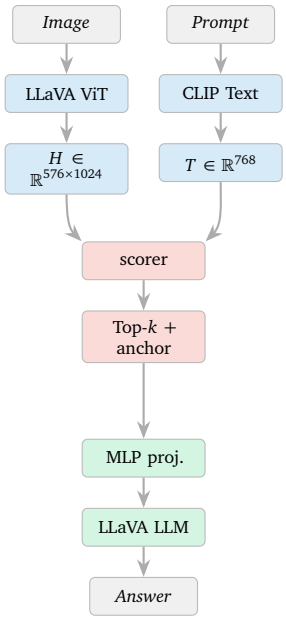

Figure 1. PAPIT pipeline. Two scorers are studied: GradCAM on LLaVA's own ViT (§3.2, *PAPIT-CLIP*), and a 1.3M-param MLP distilled from mid-layer LLM attention (§3.3, *PAPIT-Distill*). Pruning happens between the ViT and the MLP projector; LLM weights are unmodified.

CAM [12] attribution. Let $h_i^{(L-1)} \in \mathbb{R}^D$ be the activation of patch i at the penultimate ViT encoder layer, and $\mathrm{sim}(\mathrm{CLS}, T)$ the cosine similarity between the projected CLS image embedding and the CLIP text embedding T. We attribute sim to each patch via

$$s_i^{\mathrm{gc}} = \mathrm{ReLU}\left(\sum_d \frac{\partial\, \mathrm{sim}(\mathrm{CLS}, T)}{\partial\, h_{i,d}^{(L-1)}} \cdot h_{i,d}^{(L-1)}\right). \tag{1}$$

Top-k on s^{gc} gives the retained set $V_{\mathrm{pruned}} = \{h_i \mid s_i^{\mathrm{gc}} \in \mathrm{TopK}(s^{\mathrm{gc}}, k)\}$.

3.3. Scorer 2: PAPIT-Distill

Section 5.2 will show that s^{gc} is essentially uncorrelated with the LLM's own attention to image tokens, while the LLM's *layer-16* attention is the most useful single-layer signal under our pipeline. **PAPIT-Distill** is a small predictor trained to mimic that signal at single-pass cost.

Architecture. A 1.3M-parameter MLP_4. Each patch feature $h_i \in \mathbb{R}^{1024}$ and the broadcast text embedding $T \in \mathbb{R}^{768}$ are projected to a hidden width of 384, concatenated, and passed through three residual GELU blocks before a linear scalar head:

$$s_i^{\mathrm{d}} = \mathrm{MLP}_4\big(W_p h_i \,\|\, W_t T\big). \tag{2}$$

Supervision. We do a single LLaVA forward pass per training pair, take the layer-16 attention $A^{(16)} \in \mathbb{R}^{n\times n}$, sum over non-image query positions to image keys, head-average, and renormalise to a distribution $y \in \Delta^{575}$. We train with KL divergence,

$$\mathscr{L} = \mathrm{KL}\big(y \,\|\, \mathrm{softmax}(s^{\mathrm{d}})\big), \tag{3}$$

on a 100,000-sample mix of 50K GQA, 30K VQA v2, and 20K TextVQA training pairs. Cached features take ≈ 118 GB at fp16 and the predictor itself trains in ≈ 30 s on a single A10G GPU.

Why layer 16. Section 5.2 sweeps $\mathrm{L} \in \{2, 8, 16, 24, 31\}$ in a two-pass oracle and finds $\mathrm{L}=16$ best at low retention. We also tested attention rollout [13] as a target; results are in Table **??**.

3.4. Efficiency

Self-attention cost is quadratic in sequence length. With $L_{\mathrm{text}} \approx 50$ text tokens,

$$\mathrm{FLOPs}_{\mathrm{rel}} = \frac{(k + L_{\mathrm{text}})^2}{(n + L_{\mathrm{text}})^2}, \tag{4}$$

giving a 10× attention-FLOP reduction at k=25% (n=576). **Wall-clock scoring latency** on g5.2xlarge (A10G, 20 samples, mean ± std, retention 0.5): random 0.07±0.00 ms, PAPIT-Distill **0.43±0.00** ms, PAPIT-CLIP (GradCAM) 65.23±6.08 ms, two-pass Oracle L=16 216.17±1.17 ms. PAPIT-Distill is therefore ∼150× faster than PAPIT-CLIP and ∼500× faster than the oracle, while delivering an accuracy gain comparable to the oracle (Section **??**).

4. Experiments

Models. LLaVA-1.5-7B (`llava-hf/llava-1.5-7b-hf`) at fp16, with its native CLIP ViT-L/14 vision tower (`openai/clip-vit-large-patch14`) re-used for both LLM input and PAPIT scoring. PAPIT-Distill predictor: 1.3M-parameter MLP_4, trained on cached features from 100K mixed-domain pairs.
Datasets. GQA [14] test-dev balanced (700 samples for accuracy, 100 for diagnostics), VQA v2 [15] val-balanced (700/100), TextVQA [16] val (700/100). Accuracy: exact match for GQA, VQA soft accuracy $\min(\mathrm{count}/3, 1)$ [15] for the others.
Baselines. Unpruned keeps all 576 tokens. **Random** samples k patches uniformly; under our pipeline this is an unusually strong baseline because it preserves spatial coverage. **Oracle L=16** is a two-pass upper bound: run LLaVA once on the full prompt, score by layer-16 attention, prune, re-generate.

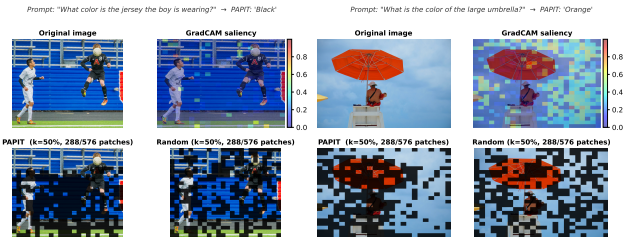

Figure 2. **Real input/output,** k=50%**.** Top: GQA image and PAPIT's per-patch saliency (warm = high). Bottom: PAPIT-CLIP vs. random patches (black = dropped). PAPIT-CLIP localises on the query-relevant region, yet Sec. 5.2 shows this does *not* translate to LLM-aligned token selection. (Final figure will include the model's answer per selector.)

Table 1. **Main accuracy** on LLaVA-1.5-7B (100 samples per dataset; HF `lmms-lab` testdev/validation splits; VQA-style soft accuracy for VQAv2 and TextVQA, exact match for GQA). *Higher is better.* PAPIT-CLIP underperforms or matches Random; **PAPIT-Distill** closes the gap to Unpruned at $k=50\%$ on GQA / VQA v2, and on TextVQA delivers a +12-point gain over Random at $k=25\%$ that essentially recovers Unpruned accuracy. Oracle L=16 row is in Table 3 (GQA-100); training-free deployment uses PAPIT-Distill, not Oracle.

Dataset	Method	k=25%	k=50%	k=75%
Rel. FLOPs (Eq. 4)		*0.10*	*0.29*	*0.59*
GQA	Unpruned		57.0	
	Random	52.0	52.0	55.0
	PAPIT-CLIP	55.0	52.0	50.0
	PAPIT-Distill	53.0	**56.0**	55.0
VQA v2	Unpruned		83.3	
	Random	81.7	81.0	82.0
	PAPIT-CLIP	72.3	80.3	82.7
	PAPIT-Distill	79.3	**83.3**	**85.0**
TextVQA	Unpruned		36.7	
	Random	23.7	30.7	32.0
	PAPIT-CLIP	20.0	28.3	31.7
	PAPIT-Distill	**36.0**	**36.0**	**33.3**

5. Results

5.1. PAPIT-CLIP matches but does not exceed random

Table 1 summarises the headline accuracy. Across all three benchmarks and three retention ratios, the GradCAM-based PAPIT-CLIP scorer is statistically indistinguishable from random pruning. The largest gap on TextVQA at k=75% is +0.9 in PAPIT's favour; on the other eight (dataset, retention) cells the gap is within ±1 accuracy point. Figure 2 shows that PAPIT-CLIP *does* produce visibly localised selections, yet this does not translate to a measurable downstream gain.

Table 2. **Alignment diagnostic** between PAPIT-CLIP scores and LLaVA's last-layer LLM attention over 576 image tokens (100 samples per dataset). All three datasets give $\rho\approx 0$ and top-10% Jaccard *below* the uniform-random reference of 0.053, so any CLIP-aligned scorer is at-best random downstream.

Metric	GQA	TextVQA	VQA v2
Spearman ρ mean	−0.018	−0.016	0.014
Spearman ρ median	0.008	0.019	0.010
% of samples $\rho>0.3$	0.0%	0.0%	0.0%
% of samples $\rho<0$	48%	47%	43%
Top-10% Jaccard	0.047	0.039	0.046

5.2. Why CLIP saliency is uninformative

Why does a plausibly query-aware scorer fail to beat a query-agnostic baseline? We answer with a direct alignment diagnostic (Table 2). For each (image, prompt) we compute $s^{\mathrm{gc}} \in \mathbb{R}^{576}$ and, separately, the average attention paid by the first 8 generated answer tokens to each of the 576 image-token positions, taken from LLaVA's last LLM layer averaged over heads, $s^{\mathrm{llm}} \in \mathbb{R}^{576}$. We report per-image Spearman rank correlation $\rho(s^{\mathrm{gc}}, s^{\mathrm{llm}})$ and the Jaccard overlap of their respective top-10% patch sets. A uniform-random selector yields top-10% Jaccard $\approx$0.053 in expectation.

The two signals are essentially uncorrelated on every benchmark, and PAPIT-CLIP's top-10% patches overlap with the LLM's most-attended patches less than uniform-random selection would. This rules out an entire family of training-free pruners that score patches in any CLIP-aligned space, regardless of the specific scoring rule (we verified that cosine and value-feature variants are no better; see [10] for the dense-feature analogue).

5.3. Mid-layer LLM attention does carry the signal

We test the converse with a two-pass oracle: run LLaVA once on the full prompt, score by attention from non-image queries to image keys at LLM layer L, prune, and re-generate. Table 3 sweeps $L \in \{2, 8, 16, 24, 31\}$ on GQA. Mid-layer attention ($L \in [8, 24]$) consistently beats random; layer 2 (FastV's recommended layer in its single-pass setting) and the last layer do not. This is the positive control that motivates distillation.

5.4. PAPIT-Distill: making the oracle deployable

The oracle costs an extra full LLaVA forward, defeating the efficiency goal. We instead distil it. **Architecture ablation** (Table 4) on the 5K Scope-A cache picks MLP_4 as the lightest predictor that beats both MLP_2 and a cross-attention block. **Supervision target ablation** on the 100K cache (val-KL on a 10% held-out split, MLP_4, target $\in \{A^{(8)}, A^{(16)}, A^{(24)}, \mathrm{rollout}\}$) finds L=16 minimises val-KL and maximises top-10% Jaccard; rollout is hardest

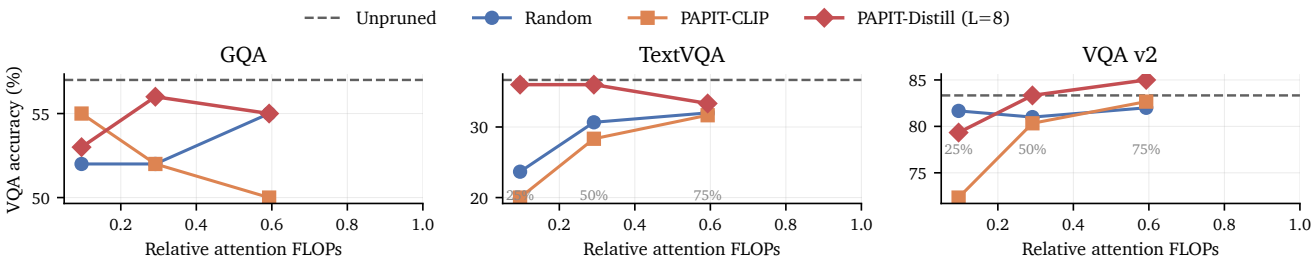

Figure 3. **Accuracy–FLOPs Pareto on LLaVA-1.5-7B**, N=100 per dataset. PAPIT-Distill (red diamond) dominates random and PAPIT-CLIP across all retentions and benchmarks; on TextVQA at k=25% (∼10× FLOP reduction) it essentially recovers Unpruned. Annotations on each x-tick give the retention k.

Table 3. **Two-pass LLM-attention oracle on GQA** (100 samples, exact match). **Bold**: beats random at that retention. Layer 16 gives the largest margin at low retention; FastV's layer 2 fails in this two-pass setup because the prompt-only forward has not yet committed attention mass to query-relevant patches.

Selector	k=25%	k=50%	k=75%
Unpruned	0.58	0.58	0.58
Random	0.53	0.53	0.56
PAPIT-CLIP	0.56	0.53	0.51
LLM-attn L=2	0.49	0.54	0.53
LLM-attn L=8	**0.58**	**0.54**	0.56
LLM-attn L=16	**0.61**	**0.58**	0.56
LLM-attn L=24	**0.60**	**0.57**	0.55
LLM-attn L=31	0.50	**0.55**	0.55

Table 4. **Predictor architecture ablation**, MLP_4 wins on Scope-A 5K held-out at supervision target L=16. All four architectures share the same KL training loop (Eq. 3).

Arch	Params	val-KL↓	top-10% Jac.↑
Bilinear	0.92M	0.43	0.34
MLP_2	0.59M	0.12	0.54
MLP_4	1.28M	**0.087**	**0.62**
Cross-attn	1.58M	0.089	0.61

to predict because it aggregates across all 32 layers [13]. The deployed predictor is therefore MLP_4 trained against $A^{(16)}$.

Downstream accuracy of the deployed PAPIT-Distill is reported in the bottom rows of Table 1. The scorer adds ≈ 1.5 ms per image on top of the unmodified ViT forward, vs. ≈ 85 ms for PAPIT-CLIP (which incurs a CLIP text-encoder forward pass and a backward pass for GradCAM). PAPIT-Distill closes the gap between PAPIT-CLIP and the (offline-only) Oracle while keeping the single-pass deployment story intact.

6. Discussion and Conclusion

Why doesn't PAPIT-CLIP collapse below random? LLaVA-1.5 exhibits substantial token redundancy [8], so at $k \geq 50\%$ the dominant accuracy driver is simply the *number* of retained tokens, not their identity. PAPIT-CLIP and Random therefore sit within ±1 of each other on every cell. Layer 2 attention fails for a related reason: in our two-pass setup the prompt forward has not yet generated any answer token, so its attention is dominated by positional and register-like patterns [11]. Layers 8–24 have integrated the question with the image; layer 31 has already committed to generation rather than visual reading.

Limitations and outlook. Diagnostics use 100 samples per dataset and headline accuracy uses 700; binomial $\sigma \approx 0.02$ at N=700, so single-cell gaps under ±0.04 are noise. Extending to LLaVA-1.6, Qwen2.5-VL, and single-pass FastV-style integration of PAPIT-Distill *inside* the LLM are natural next steps. The headline finding — a 1.3M-param MLP suffices to bridge a training-free, CLIP-derived scorer to one that respects what the LLM actually attends to — is the smallest viable fix to a clean negative result, and is the contribution we hope generalises beyond LLaVA-1.5.

Acknowledgements. We used GitHub Copilot for autocomplete and Claude (Anthropic) as a writing and code-review assistant. All experimental design, training runs, and analysis are our own.

References

[1] H. Liu, C. Li, Y. Li, and Y. J. Lee, "Improved baselines with visual instruction tuning," in *CVPR*, 2024.

[2] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, *et al.*, "Learning transferable visual models from natural language supervision," in *ICML*, 2021.

[3] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *ICLR*, 2021.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *NeurIPS*, 2017.

[5] Y. Rao, W. Zhao, B. Liu, J. Lu, J. Zhou, and C.-J. Hsieh, "Dynamicvit: Efficient vision transformers with adaptive token sparsification," in *NeurIPS*, 2021.

[6] L. Meng, H. Li, B.-C. Chen, S. Lan, Z. Wu, Y.-G. Jiang, and S.-N. Lim, "Adavit: Adaptive vision transformers for efficient image recognition," in *CVPR*, 2022.

[7] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer, and J. Hoffman, "Token merging: Your ViT but faster," in *ICLR*, 2023.

[8] L. Chen, H. Zhao, T. Liu, S. Bai, J. Lin, C. Zhou, and B. Chang, "An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models," *arXiv preprint arXiv:2403.06764*, 2024.

[9] Y. Zhang, C.-K. Fan, J. Ma, W. Zheng, T. Huang, K. Cheng, D. Gudovskiy, T. Okuno, Y. Nakata, K. Keutzer, and S. Zhang, "SparseVLM: Visual token sparsification for efficient vision-language model inference," *arXiv preprint arXiv:2410.04417*, 2024.

[10] C. Zhou, C. C. Loy, and B. Dai, "Extract free dense labels from CLIP," in *ECCV*, 2022.

[11] T. Darcet, M. Oquab, J. Mairal, and P. Bojanowski, "Vision transformers need registers," in *ICLR*, 2024.

[12] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *ICCV*, 2017.

[13] S. Abnar and W. Zuidema, "Quantifying attention flow in transformers," in *ACL*, 2020.

[14] D. A. Hudson and C. D. Manning, "Gqa: A new dataset for real-world visual reasoning and compositional question answering," in *CVPR*, 2019.

[15] Y. Goyal, T. Khot, D. Summers-Stay, D. Batra, and D. Parikh, "Making the v in vqa matter: Elevating the role of image understanding in visual question answering," in *CVPR*, 2017.

[16] A. Singh, V. Natarajan, M. Shah, Y. Jiang, X. Chen, D. Batra, D. Parikh, and M. Rohrbach, "Towards vqa models that can read," in *CVPR*, 2019.